AA 273 Midterm with Solutions

The midterm can be taken at any time during the 24 hour period it is available but must be completed within **a single 5 hour period**. The exam must be submitted no later than 3:30 PM (Pacific) on Friday, May 2nd through Gradescope.

Allowed resources

- Canvas (e.g., homework solutions, lecture notes/videos, starter code, past Ed posts)
- Textbooks
- Online documentation for any libraries that have been included in starter code (homework and exam). Examples include Numpy, Scipy, Matplotlib, Open3D, Pytorch, Pandas and standard Python libraries (math, csv).

Resources that are **NOT allowed**

- Material from previous iterations of the class (e.g., homework or exams)
- Discussion with classmates or anyone outside of the class
- The internet (except for documentation for the aforementioned libraries)
- Generative tools (including integrated coding tools like Copilot).
- You are not allowed to post questions relating to the midterm publicly on Ed between May 2 and May 5.

Additional Notes

- We prefer typeset solutions. However, if you choose to handwrite your solutions, the answers must be legible.
- Some questions require coding. Please provide associated code in your submission.
- If there are any questions regarding the exam, please direct them to the CAs through Ed as a private message or through email.

Q1 (20 points)

Bayes' Rule and Conditional Independence

You are working with a new medical sensor that can sense a subset of human emotion. Specifically, it can detect happiness, given by random variable X_1, and calmness, given by random variable X_2. $X_1 = 1$ indicates that a person is happy and $X_1 = 0$ indicates that the person is sad. Similarly, $X_2 = 1$ indicates that the person is calm, $X_2 = 0$ indicates that the person is agitated/excited. **Do NOT assume X_1 and X_2 are independent**. Table 1 gives the prior joint probability of X_1 and X_2.

This emotion sensor's measurement is given by Y and has two modes: green ($Y = 1$) or red ($Y = 0$). Table 2 gives the probabilities of the sensor's output given the person's emotional state. **Assume the measurement $Y^{(t)}$ at each time t is conditionally independent, conditioned on the states X_1 and X_2.**

Note: We do not consider natural dynamics of the system here (i.e., the person's emotion never changes), and there is no predict step.

X_1	X_2	$P(X_1, X_2)$
0	0	0.2
0	1	0.2
1	0	0.3
1	1	0.3

Table 1: Prior joint probability of X_1 and X_2

X_1	X_2	Y	$P(Y \mid X_1, X_2)$
0	0	0	0.9
0	0	1	0.1
0	1	0	0.8
0	1	1	0.2
1	0	0	0.4
1	0	1	0.6
1	1	0	0.05
1	1	1	0.95

Table 2: Sensor model $P(Y \mid X_1, X_2)$

(a) Express $P(Y = 1)$ as a function of known values, and compute the initial value using the prior values for X_1 and X_2.

Solution:

$$\begin{aligned} P(Y=1) &= \sum_{X_1,X_2} P(Y=1 \mid X_1, X_2) P(X_1, X_2) \\ &= P(Y=1 \mid X_1=0, X_2=0) P(X_1=0, X_2=0) + \\ &\quad P(Y=1 \mid X_1=0, X_2=1) P(X_1=0, X_2=1) + \\ &\quad P(Y=1 \mid X_1=1, X_2=0) P(X_1=1, X_2=0) + \\ &\quad P(Y=1 \mid X_1=1, X_2=1) P(X_1=1, X_2=1) \\ &= 0.1(0.2) + 0.2(0.2) + 0.6(0.3) + 0.95(0.3) \\ &= 0.5250 \end{aligned}$$

(b) You receive one measurement $Y^{(1)} = \text{Green}$. What are your new probabilities $P(X_1 = 1 \mid Y^{(1)} = 1)$ and $P(X_2 = 1 \mid Y^{(1)} = 1)$?

Solution:

Using Bayes' rule:

$$P(X_1 = 1|Y = 1) = \frac{P(X_1 = 1, Y = 1)}{P(Y = 1)}$$

$$\begin{aligned}
P(X_1 = 1, Y = 1) &= \sum_{x_2 \in \{0,1\}} P(Y = 1|X_1 = 1, X_2 = x_2) \cdot P(X_1 = 1, X_2 = x_2) && (1)\\
&= P(Y = 1|X_1 = 1, X_2 = 0) \cdot P(X_1 = 1, X_2 = 0) && (2)\\
&\quad + P(Y = 1|X_1 = 1, X_2 = 1) \cdot P(X_1 = 1, X_2 = 1) && (3)\\
&= 0.6 \times 0.3 + 0.95 \times 0.3 && (4)\\
&= 0.18 + 0.285 && (5)\\
&= 0.465 && (6)
\end{aligned}$$

$$P(X_1 = 1|Y = 1) = \frac{0.465}{0.525} = \frac{31}{35} \approx 0.886$$

Similarly, $P(X_2 = 1|Y = 1)$:

$$P(X_2 = 1|Y = 1) = \frac{P(X_2 = 1, Y = 1)}{P(Y = 1)}$$

$$\begin{aligned}
P(X_2 = 1, Y = 1) &= \sum_{x_1 \in \{0,1\}} P(Y = 1|X_1 = x_1, X_2 = 1) \cdot P(X_1 = x_1, X_2 = 1) && (7)\\
&= P(Y = 1|X_1 = 0, X_2 = 1) \cdot P(X_1 = 0, X_2 = 1) && (8)\\
&\quad + P(Y = 1|X_1 = 1, X_2 = 1) \cdot P(X_1 = 1, X_2 = 1) && (9)\\
&= 0.2 \times 0.2 + 0.95 \times 0.3 && (10)\\
&= 0.04 + 0.285 && (11)\\
&= 0.325 && (12)
\end{aligned}$$

$$P(X_2 = 1|Y = 1) = \frac{0.325}{0.525} = \frac{13}{21} \approx 0.619$$

(c) Generate the probability over time from $t_0 = 0$ to $t_f = 400$ of $P(X_1 = 1 \mid Y^{(1:t_f)})$ (being happy) and $P(X_2 = 1)$ (being calm), given the following measurement history:

- $Y^{(t)}$ = Green from $t = 1$ to $t = 100$
- $Y^{(t)}$ = Red from $t = 101$ to $t = 400$

Show $P(X_1 = 1 \mid Y^{(1:t_f)})$ and $P(X_2 = 1 \mid Y^{(1:t_f)})$ over time on the same plot, with clear axes labels, line labels, and a legend. You may use your coding language of preference. Submit your plot and your code.

```
import numpy as np
import matplotlib.pyplot as plt

# Initial prior joint distribution
# P(X1, X2) where indices are (X1, X2)
initial_joint = np.array([[0.2, 0.2],   # P(X1=0, X2=0), P(X1=0, X2=1)
                          [0.3, 0.3]])   # P(X1=1, X2=0), P(X1=1, X2=1)

# Sensor model P(Y | X1, X2)
# First index is Y, second and third are X1, X2
sensor_model = np.zeros((2, 2, 2))
# Y = 0 (Red)
sensor_model[0, 0, 0] = 0.9   # P(Y=0 | X1=0, X2=0)
sensor_model[0, 0, 1] = 0.8   # P(Y=0 | X1=0, X2=1)
sensor_model[0, 1, 0] = 0.4   # P(Y=0 | X1=1, X2=0)
sensor_model[0, 1, 1] = 0.05  # P(Y=0 | X1=1, X2=1)
# Y = 1 (Green)
sensor_model[1, 0, 0] = 0.1   # P(Y=1 | X1=0, X2=0)
sensor_model[1, 0, 1] = 0.2   # P(Y=1 | X1=0, X2=1)
sensor_model[1, 1, 0] = 0.6   # P(Y=1 | X1=1, X2=0)
sensor_model[1, 1, 1] = 0.95  # P(Y=1 | X1=1, X2=1)

def update_joint_distribution(joint_prior, observation, sensor_model):
    """
    Update joint distribution using Bayes' rule
    """
    # Compute likelihood P(Y | X1, X2) * P(X1, X2)
    likelihood = sensor_model[observation] * joint_prior
    # Normalize to get posterior
    posterior = likelihood / np.sum(likelihood)
    return posterior

def compute_marginals(joint_dist):
    """
    Compute marginal probabilities from joint distribution
    """
    # P(X1 = 1) = P(X1=1, X2=0) + P(X1=1, X2=1)
    p_x1_1 = joint_dist[1, 0] + joint_dist[1, 1]
    # P(X2 = 1) = P(X1=0, X2=1) + P(X1=1, X2=1)
    p_x2_1 = joint_dist[0, 1] + joint_dist[1, 1]
    return p_x1_1, p_x2_1

# Initialize arrays to store probabilities over time
time_steps = 401  # 0 to 400
p_x1_1_history = np.zeros(time_steps)
p_x2_1_history = np.zeros(time_steps)
# Start with initial prior
current_joint = initial_joint.copy()
# Compute initial marginals
p_x1_1_history[0], p_x2_1_history[0] = compute_marginals(current_joint)

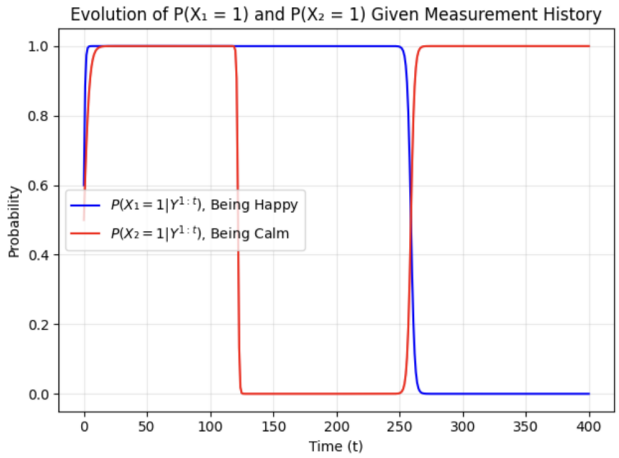
# Update for each time step
for t in range(1, time_steps):
    # Determine observation at time t
    if 1 <= t <= 100:
        observation = 1  # Green
    else:  # 101 <= t <= 400
        observation = 0  # Red
    # Update joint distribution
    current_joint = update_joint_distribution(current_joint, observation, sensor_model)
    # Compute marginals
    p_x1_1_history[t], p_x2_1_history[t] = compute_marginals(current_joint)

# Create the plot
plt.plot(range(time_steps), p_x1_1_history, 'b-', label=r'$P(X_1 = 1 | Y^{1:t})$, Being Happy')
plt.plot(range(time_steps), p_x2_1_history, 'r-', label=r'$P(X_2 = 1 | Y^{1:t})$, Being Calm')

# Add labels and formatting
plt.xlabel('Time (t)')
plt.ylabel('Probability')
plt.title(r'Evolution of $P(X_1 = 1)$ and $P(X_2 = 1)$ Given Measurement History')
plt.legend(loc='center left')
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```

Alternative Code Solution:

Different way to formulate the problem that may be less intuitive, but significantly more concise. The resultant plot is the same.




```
    # %%
import numpy as np
import matplotlib.pyplot as plt

# %%
M_Y1 = np.array([[0.1, 0.2], [0.6, 0.95]])
M_Y0 = 1 - M_Y1

# %%
px1x2_prior = np.array([[0.2, 0.2], [0.3, 0.3]])

# %%
total_t = 400
px1x2_hist = np.zeros((total_t, 2, 2))
px1x2_hist[0, :, :] = px1x2_prior
for t in range(1, total_t):
    if t < 100:
        meas = M_Y1
    else:
        meas = M_Y0
    update = meas * px1x2_hist[t-1, :, :]
    px1x2_hist[t, :, :] = update/np.sum(update)


# %%
p_x1_hist = np.sum(px1x2_hist, axis=2)[:, 1]
p_x2_hist = np.sum(px1x2_hist, axis=1)[:, 1]

# %%
plt.plot(np.arange(total_t), p_x1_hist, label='P(Happy)')
plt.plot(np.arange(total_t), p_x2_hist, label='P(Calm)')
plt.legend()
plt.xlabel('Time')
plt.ylabel('Probability')
```

Q2 [20 points]

Expectations, Means, Covariance, Moments, and related Statistical Concepts

Problem Setup: Consider an autonomous vehicle that is trying to estimate its position (denoted as X) using two different sensors. The true position, X, is unknown, and the vehicle receives two measurements: Z_1 and Z_2. Each sensor measurement consists of the true position plus some measurement noise:

$$Z_1 = X + V_1 \quad \text{and} \quad Z_2 = X + V_2$$

Where: $V_1 \sim \mathcal{N}(0, \sigma_1^2)$, $V_2 \sim \mathcal{N}(0, \sigma_2^2)$, and X, V_1, and V_2 are assumed to be mutually independent of each other.

You are tasked with developing an optimal linear estimator of X based on the two sensor measurements Z_1 and Z_2.

(a) Compute the means $\mathbb{E}[Z_1], \mathbb{E}[Z_2]$ and variances $\text{Var}(Z_1), \text{Var}(Z_2)$ of the two sensor measurements.

(b) Compute the covariance between the two measurements $\text{Cov}(Z_1, Z_2)$.

(c) Derive the expected value of the estimator $\hat{Z} = \alpha Z_1 + \beta Z_2$. What should be the relationship between α and β so that $\hat{Z}$ is an unbiased estimator of X. Hint: An estimator $\hat{\theta}$ of a parameter θ is called unbiased if $\mathbb{E}[\hat{\theta}] = \theta$, where $\mathbb{E}[\hat{\theta}]$ denotes the expected value of the estimator.

(d) What are the values of α and β so that $\hat{Z}$ has minimum variance and is an unbiased estimator of X? What is this minimum variance?

(e) Let this unbiased estimator of X with minimum variance be Z^*. Define the estimation error as $e = Z^* - X$

(i) What is the mean of this error $\mathbb{E}[e]$?

(ii) What is the variance of the error $\text{Var}(e)$?

(iii) Based on the noise characteristics of measurements, describe the probability distribution of the error e. What are the parameters of this distribution?

(iv) Using the distribution calculate the probability that the absolute error exceeds three standard deviations, i.e., compute: $P(|e| > 3\sigma_e)$

Express your answer in terms of the standard Gaussian cumulative distribution function: $\Phi(y) = P(Y \leq y)$, where $Y \sim \mathcal{N}(0, 1)$.

(f) Explain your intuition behind the obtained values of α and β for Z^*. Hint: What happens when one sensor is noisier than the other?

Solution:

Given:

- $Z_1 = X + V_1$ and $Z_2 = X + V_2$
- $V_1 \sim \mathcal{N}(0, \sigma_1^2)$, $V_2 \sim \mathcal{N}(0, \sigma_2^2)$
- X, V_1, and V_2 are mutually independent

(a) Means and variances of sensor measurements For $Z_1 = X + V_1$:

$$\mathbb{E}[Z_1] = \mathbb{E}[X] + \mathbb{E}[V_1] = \mathbb{E}[X] + 0 = \mathbb{E}[X] \tag{13}$$

$$\text{Var}(Z_1) = \text{Var}(X) + \text{Var}(V_1) = \text{Var}(X) + \sigma_1^2 \tag{14}$$

For $Z_2 = X + V_2$:

$$\mathbb{E}[Z_2] = \mathbb{E}[X] + \mathbb{E}[V_2] = \mathbb{E}[X] + 0 = \mathbb{E}[X] \tag{15}$$
$$\text{Var}(Z_2) = \text{Var}(X) + \text{Var}(V_2) = \text{Var}(X) + \sigma_2^2 \tag{16}$$

(b) Covariance between measurements

$$\begin{aligned} \text{Cov}(Z_1, Z_2) &= \text{Cov}(X + V_1, X + V_2) && (17)\\ &= \text{Cov}(X, X) + \text{Cov}(X, V_2) + \text{Cov}(V_1, X) + \text{Cov}(V_1, V_2) && (18)\\ &= \text{Var}(X) + 0 + 0 + 0 = \text{Var}(X) && (19) \end{aligned}$$

(c) Expected value and unbiasedness condition For $\hat{Z} = \alpha Z_1 + \beta Z_2$:

$$\mathbb{E}[\hat{Z}] = \alpha\mathbb{E}[Z_1] + \beta\mathbb{E}[Z_2] = (\alpha + \beta)\mathbb{E}[X] \tag{20}$$

For unbiasedness, we need $\mathbb{E}[\hat{Z}] = \mathbb{E}[X]$, which requires:

$$\boxed{\alpha + \beta = 1} \tag{21}$$

(d) Optimal values for minimum variance Given the constraint $\alpha + \beta = 1$, we can write $\beta = 1 - \alpha$.

The variance of the estimator is:

$$\begin{aligned} \text{Var}(\hat{Z}) &= \text{Var}(\alpha Z_1 + \beta Z_2) && (22)\\ &= \alpha^2\text{Var}(Z_1) + \beta^2\text{Var}(Z_2) + 2\alpha\beta\text{Cov}(Z_1, Z_2) && (23)\\ &= \alpha^2(\text{Var}(X) + \sigma_1^2) + (1-\alpha)^2(\text{Var}(X) + \sigma_2^2) + 2\alpha(1-\alpha)\text{Var}(X) && (24) \end{aligned}$$

After simplification, the variance becomes:

$$\text{Var}(\hat{Z}) = \text{Var}(X) + \alpha^2\sigma_1^2 + (1-\alpha)^2\sigma_2^2 \tag{25}$$

To minimize, we take the derivative with respect to α and set to zero:

$$\frac{d}{d\alpha}[\alpha^2\sigma_1^2 + (1-\alpha)^2\sigma_2^2] = 2\alpha\sigma_1^2 - 2(1-\alpha)\sigma_2^2 = 0 \tag{26}$$

Solving for α and β:

$$\boxed{\alpha = \frac{\sigma_2^2}{\sigma_1^2 + \sigma_2^2}, \quad \beta = \frac{\sigma_1^2}{\sigma_1^2 + \sigma_2^2}} \tag{27}$$

The minimum variance is:

$$\boxed{\text{Var}(\hat{Z}) = \text{Var}(X) + \frac{\sigma_1^2\sigma_2^2}{\sigma_1^2 + \sigma_2^2}} \tag{28}$$

(e) Error analysis for optimal estimator With the optimal weights, the error is:

$$e = Z^* - X = \frac{\sigma_2^2}{\sigma_1^2 + \sigma_2^2}V_1 + \frac{\sigma_1^2}{\sigma_1^2 + \sigma_2^2}V_2 \tag{29}$$

(i) Mean of error

$$\boxed{\mathbb{E}[e] = 0} \tag{30}$$

(ii) Variance of error

$$\text{Var}(e) = \left(\frac{\sigma_2^2}{\sigma_1^2+\sigma_2^2}\right)^2 \sigma_1^2 + \left(\frac{\sigma_1^2}{\sigma_1^2+\sigma_2^2}\right)^2 \sigma_2^2 \tag{31}$$

$$= \boxed{\frac{\sigma_1^2\sigma_2^2}{\sigma_1^2+\sigma_2^2}} \tag{32}$$

(iii) Error distribution Since e is a linear combination of normal random variables:

$$\boxed{e \sim \mathcal{N}\left(0, \frac{\sigma_1^2\sigma_2^2}{\sigma_1^2+\sigma_2^2}\right)} \tag{33}$$

(iv) Probability calculation The standard deviation of the error is $\sigma_e = \sqrt{\frac{\sigma_1^2\sigma_2^2}{\sigma_1^2+\sigma_2^2}}$.

For a standard normal $Y \sim \mathcal{N}(0,1)$:

$$\boxed{P(|e| > 3\sigma_e) = 2[1-\Phi(3)]} \tag{34}$$

(f) The optimal weights are inversely proportional to the noise variances. When one sensor is noisier than the other (larger σ^2), the optimal estimator assigns a smaller weight to that sensor. This makes intuitive sense: we should trust the more accurate sensor more heavily. The estimator effectively performs a weighted average where the weights reflect the relative precision of each sensor.

Q3 (20 points)

Dependence and Correlation

(a) Let $X = [X_1 \quad X_2]^T$ be a random variable described by a multivariate Gaussian distribution, $X \sim \mathcal{N}(\mu, \Sigma)$, where

$$\mu = \begin{bmatrix} \mu_1 \\ \mu_2 \end{bmatrix}, \qquad \Sigma = \begin{bmatrix} \Sigma_{11} & \Sigma_{12} \\ \Sigma_{21} & \Sigma_{22} \end{bmatrix}$$

Show that if $\Sigma_{12} = \mathbb{E}[(X_1 - \mu_1)(X_2 - \mu_2)^T] = 0$, X_1 and X_2 are independent.

Solution:

For a multivariate Gaussian distribution $X \sim \mathcal{N}(\mu, \Sigma)$:

$$f(X) = \frac{1}{(2\pi)^{n/2}|\Sigma|^{1/2}} e^{-\frac{1}{2}(x-\mu)^T\Sigma^{-1}(x-\mu)} \tag{35}$$

where:

$$x = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}, \quad \mu = \begin{bmatrix} \mu_1 \\ \mu_2 \end{bmatrix}, \quad \Sigma = \begin{bmatrix} \Sigma_{11} & 0 \\ 0 & \Sigma_{22} \end{bmatrix} \tag{36}$$

$$\Sigma^{-1} = \begin{bmatrix} A & B \\ C & D \end{bmatrix} \tag{37}$$

$$\begin{bmatrix} \Sigma_{11} & 0 \\ 0 & \Sigma_{22} \end{bmatrix} \begin{bmatrix} A & B \\ C & D \end{bmatrix} = \begin{bmatrix} I & 0 \\ 0 & I \end{bmatrix} \tag{38}$$

$$\Sigma_{11}A = I, \quad A = \Sigma_{11}^{-1} \tag{39}$$

$$\Sigma_{22}D = I, \quad D = \Sigma_{22}^{-1} \tag{40}$$

$$\Sigma^{-1} = \begin{bmatrix} \Sigma_{11}^{-1} & 0 \\ 0 & \Sigma_{22}^{-1} \end{bmatrix} \tag{41}$$

Aside:

$$|\Sigma| = |\Sigma_{11}||\Sigma_{22}| \tag{42}$$

Therefore:

$$\frac{1}{(2\pi)^{\frac{n_1}{2}}(2\pi)^{\frac{n_2}{2}}|\Sigma_{11}|^{\frac{1}{2}}|\Sigma_{22}|^{\frac{1}{2}}} e^{-\frac{1}{2}(x-\mu)^T\Sigma^{-1}(x-\mu)} \tag{43}$$

where, n_1 is associated with X_1 and n_2 is associated with X_2.

Now considering the other component:

$$= \begin{bmatrix} x_1 - \mu_1 \\ x_2 - \mu_2 \end{bmatrix}^T \begin{bmatrix} \Sigma_{11}^{-1} & 0 \\ 0 & \Sigma_{22}^{-1} \end{bmatrix} \begin{bmatrix} x_1 - \mu_1 \\ x_2 - \mu_2 \end{bmatrix} \tag{44}$$

$$= \begin{bmatrix} (x_1 - \mu_1)^T & (x_2 - \mu_2)^T \end{bmatrix} \begin{bmatrix} \Sigma_{11}^{-1}(x_1 - \mu_1) \\ \Sigma_{22}^{-1}(x_2 - \mu_2) \end{bmatrix} \tag{45}$$

$$= (x_1 - \mu_1)^T\Sigma_{11}^{-1}(x_1 - \mu_1) + (x_2 - \mu_2)^T\Sigma_{22}^{-1}(x_2 - \mu_2) \tag{46}$$

After substituting and expanding:

$$= \underbrace{\frac{1}{(2\pi)^{\frac{n_1}{2}}|\Sigma_{11}|^{\frac{1}{2}}} e^{-\frac{1}{2}(x_1-\mu_1)^T\Sigma_{11}^{-1}(x_1-\mu_1)}}_{f(x_1)} \underbrace{\frac{1}{(2\pi)^{\frac{n_2}{2}}|\Sigma_{22}|^{\frac{1}{2}}} e^{-\frac{1}{2}(x_2-\mu_2)^T\Sigma_{22}^{-1}(x_2-\mu_2)}}_{f(x_2)} \tag{47}$$

$$f(X) = f(X_1) \cdot f(X_2) \tag{48}$$

This shows that X_1 and X_2 are independent.

(b) Let $W \in \mathbb{R}$ be a random variable that is uniformly distributed between interval $[0, 1]$, (i.e., $p(w) = 1$ and $\int_0^1 p(w)dw = 1$). Let V_1 and V_2 in $\mathbb{R}$ be two random variables that are normally distributed. $V_1 \sim \mathcal{N}(0, \sigma_1^2), V_2 \sim \mathcal{N}(0, \sigma_2^2)$. W, V_1, V_2 are all independent from each other. Consider the following two random variables:

$$Y_1 = \sin(2\pi W) + V_1$$
$$Y_2 = \cos(2\pi W) + V_2$$

Show mathematically that Y_1 and Y_2 are uncorrelated. Are they independent, why or why not? Are they conditionally independent given W? (Only a verbal justification is required for these last two questions.)

Solution:

$$\begin{aligned}
\mathrm{Cov}(Y_1, Y_2) &= \mathbb{E}[Y_1Y_2] - \mathbb{E}[Y_1]\mathbb{E}[Y_2] && (49)\\
&= \mathbb{E}[(\sin(2\pi W) + V_1)(\cos(2\pi W) + V_2)] && (50)\\
&= \mathbb{E}[\sin(2\pi W)\cos(2\pi W)] + \mathbb{E}[\sin(2\pi W)V_2] && (51)\\
&\quad + \mathbb{E}[\cos(2\pi W)V_1] + \mathbb{E}[V_1V_2] && (52)\\
&= \mathbb{E}[\sin(2\pi W)\cos(2\pi W)] + \underbrace{\mathbb{E}[\sin(2\pi W)]\mathbb{E}[V_2]}_{=0} && (53)\\
&\quad + \underbrace{\mathbb{E}[\cos(2\pi W)]\mathbb{E}[V_1]}_{=0} + \underbrace{\mathbb{E}[V_1]\mathbb{E}[V_2]}_{=0} && (54)\\
&= \mathbb{E}[\sin(2\pi W)\cos(2\pi W)] && (55)\\
&= \int_0^1 \sin(2\pi w)\cos(2\pi w)dw && (56)\\
&= \int_0^1 \frac{1}{2}\sin(4\pi w)dw && (57)\\
&= \frac{1}{2}\left[-\frac{1}{4\pi}\cos(4\pi w)\right]_0^1 && (58)\\
&= \frac{1}{2}\left[-\frac{1}{4\pi}(\cos(4\pi) - \cos(0))\right] = 0 && (59)
\end{aligned}$$

Alternative Calculation:

$$\begin{aligned}
\mathbb{E}[f(x,y)] &= \iint f(x,y)\phi(x,y)dxdy && (60)\\
&= \mathbb{E}[\sin(2\pi w)V_1] + \mathbb{E}[\cos(2\pi w)V_2] && (61)\\
&= \int_0^1 \int \sin(2\pi w)v_1 f_W(w) f_{V_1}(v_1)dwdv_1 && (62)\\
&= \int_0^1 \sin(2\pi w)dw \cdot \int v_1 f_{V_1}(v_1)dv_1 && (63)\\
&= 0 \cdot \mathbb{E}[V_1] = 0 && (64)
\end{aligned}$$

$\text{Cov}(Y_1, Y_2) = 0 \Rightarrow Y_1, Y_2$ are uncorrelated.

Q4 (20 points)

Modifications to the Kalman Filter

You are given a linear Gaussian model:

$$X_{t+1} = A_t X_t + B_t u_t + W_t$$
$$Y_{t+1} = C_t X_t + D_t u_t + V_t$$

Where W_t, V_t are Gaussian white noise processes, uncorrelated with each other and with the initial state X_0. In class, we derived the standard Kalman filter (KF) for the linear Gaussian model with some simplifying assumptions on the noise models W_t (process noise) and V_t (measurement noise), and set the matrix $D_t = 0$. In this problem, we will relax some assumptions to see how that changes our KF equations. **You can use the Gaussian estimation equations, as well as $Cov(X_t, V_t \mid Y_{1:t-1}) = 0$ and $Cov(X_t, W_t \mid Y_{1:t}) = 0$, or any other properties deriving from the noise independence and whiteness assumptions of the linear Gaussian model.**

(a) Derive the Kalman filter equations when the state-space matrix $D_t \neq 0$. Which equations are the same and which are different from the original KF?

Solution:

Mean prediction

$$\begin{aligned}
\mu_{t+1|t} &= \mathbb{E}[X_{t+1} \mid Y_{1:t}] && (65)\\
&= \mathbb{E}[A_t X_t + B_t u_t + W_t \mid Y_{1:t}] && (66)\\
&= A_t \mathbb{E}[X_t \mid Y_{1:t}] + \mathbb{E}[B_t u_t \mid Y_{1:t}] + \mathbb{E}[W_t \mid Y_{1:t}] && (67)\\
&= A_t \mu_{t|t} + B_t u_t + 0 \quad (\text{since } B_t u_t \text{ deterministic, and } \mathbb{E}[W_t] = 0) && (68)
\end{aligned}$$

$$\boxed{\mu_{t+1|t} = A_t \mu_{t|t} + B_t u_t \text{ (same as KF)}}$$

Covariance prediction

$$\begin{aligned}
\Sigma_{t+1|t} &= \mathbb{E}\left[(X_{t+1} - \mu_{t+1|t})(X_{t+1} - \mu_{t+1|t})^\top \mid Y_{1:t}\right] && (69)\\
&= \mathbb{E}\left[(A_t(X_t - \mu_{t|t}) + W_t)(A_t(X_t - \mu_{t|t}) + W_t)^\top \mid Y_{1:t}\right] && (70)\\
&= A_t \mathbb{E}[(X_t - \mu_{t|t})(X_t - \mu_{t|t})^\top \mid Y_{1:t}] A_t^\top + \mathbb{E}[W_t W_t^\top \mid Y_{1:t}] && (71)\\
&\quad + \mathbb{E}[W_t (X_t - \mu_{t|t})^\top \mid Y_{1:t}] A_t^\top + A_t \mathbb{E}[(X_t - \mu_{t|t}) W_t^\top \mid Y_{1:t}] && (72)\\
&= A_t \Sigma_{t|t} A_t^\top + Q_t \quad (\text{since } W_t \text{ uncorrelated with } X_t \text{ and zero-mean noise}) && (73)
\end{aligned}$$

$$\boxed{\Sigma_{t+1|t} = A_t \Sigma_{t|t} A_t^\top + Q_t \text{ (same as KF)}}$$

It makes sense that the prediction steps don't change with the inclusion of D_t, since the dynamics model has not changed.

Mean update

From the Gaussian estimation equations, we have

$$\mu_{t|t} = \mu_{t|t-1} + \Sigma_{xy} \Sigma_y^{-1} (y_t - \mu_{y_t|y_{1:t-1}}) \tag{74}$$

We need to find all the quantities on the right-hand side of the equation

$$\begin{aligned}
\mu_{y_t|y_{1:t-1}} &= \mathbb{E}[Y_t \mid Y_{1:t-1}] && (75)\\
&= \mathbb{E}[C_t X_t + D_t u_t + V_t \mid Y_{1:t-1}] && (76)\\
&= C_t \mathbb{E}[X_t \mid Y_{1:t-1}] + \mathbb{E}[D_t u_t \mid Y_{1:t-1}] + \mathbb{E}[V_t \mid Y_{1:t-1}] && (77)\\
&= C_t \mu_{t|t-1} + D_t u_t && (78)
\end{aligned}$$

We can also calculate the measurement covariance Σ_y as

$$\begin{aligned}
\Sigma_y &= \text{Cov}(Y_t \mid Y_{1:t-1}) && (79)\\
&= \mathbb{E}\left[(C_t(X_t - \mu_{t|t-1}) + V_t)(C_t(X_t - \mu_{t|t-1}) + V_t)^\top \mid Y_{1:t-1}\right] && (80)\\
&= C_t\Sigma_{t|t-1}C_t^\top + C_t\text{Cov}(X_t, V_t \mid Y_{1:t-1}) && (81)\\
&\quad + \text{Cov}(V_t, X_t \mid Y_{1:t-1})C_t^\top + \text{Cov}(V_t \mid Y_{1:t-1}) && (82)\\
&= C_t\Sigma_{t|t-1}C_t^\top + R_t \quad \text{(since } V_t \text{ uncorrelated with } X_t \text{ and mean zero)} && (83)
\end{aligned}$$

Then we find the cross-covariance between x and y

$$\begin{aligned}
\Sigma_{xy} &= \text{Cov}(X_t, Y_t \mid Y_{1:t-1}) && (84)\\
&= \text{Cov}(X_t, C_tX_t + D_tu_t + V_t \mid Y_{1:t-1}) && (85)\\
&= \mathbb{E}\left[(X_t - \mu_{t|t-1})(C_t(X_t - \mu_{t|t-1}) + V_t)^\top \mid Y_{1:t-1}\right] && (86)\\
&= \Sigma_{t|t-1}C_t^\top + \text{Cov}(X_t, V_t \mid Y_{1:t-1}) && (87)\\
&= \Sigma_{t|t-1}C_t^\top \quad \text{(since } X_t \text{ and } V_t \text{ uncorrelated)} && (88)
\end{aligned}$$

Putting all these variables together, we get

$$\boxed{\begin{gathered}\mu_{t|t} = \mu_{t|t-1} - K\left(y_t - \left(C_t\mu_{t|t-1} + D_tu_t\right)\right) \text{ (different from KF)}\\ K = \Sigma_{t|t-1}C_t^\top\left(C_t\Sigma_{t|t-1}C_t^\top + R_t\right)^{-1}\end{gathered}}$$

Covariance update

From the Gaussian estimation equations:

$$\Sigma_{t|t} = \Sigma_{t|t-1} - \Sigma_{xy}\Sigma_y^{-1}\Sigma_{yx} \tag{89}$$

$$\Sigma_{t|t} = \Sigma_{t|t-1} - \Sigma_{t|t-1}C_t^\top\left(C_t\Sigma_{t|t-1}C_t^\top + R_t\right)^{-1}C_t\Sigma_{t|t-1} \tag{90}$$

$$\boxed{\Sigma_{t|t} = \Sigma_{t|t-1} - KC_t\Sigma_{t|t-1} \text{(same as KF)}}$$

(b) We previously assumed that the process noise and the measurement noise have zero mean $\mu_{W_t} = \mu_{V_t} = 0\ \forall t$. You can assume $D_t = 0$ for this part. Now, derive the Kalman filter equations when

$$\begin{aligned}W_t &\sim \text{GWN}\left(\mu_{W_t}, Q\right), \quad \mu_{W_t} \neq 0\\ V_t &\sim \text{GWN}\left(\mu_{V_t}, R\right), \quad \mu_{V_t} \neq 0\end{aligned}$$

Which equations are the same and which are different from the original KF?

Solution:

Mean prediction

$$\begin{aligned}
\mu_{t+1|t} &= \mathbb{E}[X_{t+1} \mid Y_{1:t}] && (91)\\
&= \mathbb{E}[A_tX_t + B_tu_t + W_t \mid Y_{1:t}] && (92)\\
&= A_t\mathbb{E}[X_t \mid Y_{1:t}] + \mathbb{E}[B_tu_t \mid Y_{1:t}] + \mathbb{E}[W_t \mid Y_{1:t}] && (93)\\
&= A_t\mu_{t|t} + B_tu_t + \mu_{W_t} \quad \text{(since } B_tu_t \text{ deterministic, and } \mathbb{E}[W_t] = \mu_{W_t}) && (94)
\end{aligned}$$

$$\boxed{\mu_{t+1|t} = A_t\mu_{t|t} + B_tu_t + \mu_{W_t} \text{ (different from KF)}}$$

Covariance prediction

$$\Sigma_{t+1|t} = \mathbb{E}\left[(X_{t+1} - \mu_{t+1|t})(X_{t+1} - \mu_{t+1|t})^\top \mid Y_{1:t}\right] \tag{95}$$
$$= \mathbb{E}\left[(A_t(X_t - \mu_{t|t}) + (W_t - \mu_{w_t}))(A_t(X_t - \mu_{t|t}) + (W_t - \mu_{w_t}))^\top \mid Y_{1:t}\right] \tag{96}$$
$$= A_t\mathbb{E}[(X_t - \mu_{t|t})(X_t - \mu_{t|t})^\top \mid Y_{1:t}]A_t^\top + \mathbb{E}[(W_t - \mu_{w_t})(W_t - \mu_{w_t})^\top \mid Y_{1:t}] \tag{97}$$
$$+ A_t\mathbb{E}[(X_t - \mu_{t|t})(W_t - \mu_{w_t})^\top \mid Y_{1:t}] + \mathbb{E}[(W_t - \mu_{w_t})(X_t - \mu_{t|t})^\top \mid Y_{1:t}]A_t^\top \tag{98}$$
$$= A_t\Sigma_{t|t}A_t^\top + Q_t \quad \text{(since } W_t \text{ uncorrelated with } X_t \text{ and zero-mean noise)} \tag{99}$$

$$\boxed{\Sigma_{t+1|t} = A_t\Sigma_{t|t}A_t^\top + Q_t \text{ (same as KF)}}$$

Mean update

From the Gaussian estimation equations, we have

$$\mu_{t|t} = \mu_{t|t-1} + \Sigma_{xy}\Sigma_y^{-1}(y_t - \mu_{y_t|y_{1:t-1}}) \tag{100}$$

We need to find all the quantities on the right-hand side of the equation

$$\mu_{y_t|y_{1:t-1}} = \mathbb{E}[Y_t \mid Y_{1:t-1}] \tag{101}$$
$$= \mathbb{E}[C_tX_t + V_t \mid Y_{1:t-1}] \tag{102}$$
$$= C_t\mathbb{E}[X_t \mid Y_{1:t-1}] + \mathbb{E}[V_t \mid Y_{1:t-1}] \tag{103}$$
$$= C_t\mu_{t|t-1} + \mu_{v_t} \tag{104}$$

We can also calculate the measurement covariance Σ_y as

$$\Sigma_y = \mathrm{Cov}(Y_t \mid Y_{1:t-1}) \tag{105}$$
$$= \mathbb{E}\left[(C_t(X_t - \mu_{t|t-1}) + (V_t - \mu_{v_t}))(\cdots)^\top \mid Y_{1:t-1}\right] \tag{106}$$
$$= C_t\Sigma_{t|t-1}C_t^\top + C_t\mathrm{Cov}(X_t, V_t \mid Y_{1:t-1}) \tag{107}$$
$$+ \mathrm{Cov}(V_t, X_t \mid Y_{1:t-1})C_t^\top + \mathrm{Cov}(V_t \mid Y_{1:t-1}) \tag{108}$$
$$= C_t\Sigma_{t|t-1}C_t^\top + R_t \quad \text{(since } V_t \text{ uncorrelated with } X_t) \tag{109}$$

We also find the cross-covariance Σ_{xy}

$$\Sigma_{xy} = \mathrm{Cov}(X_t, Y_t \mid Y_{1:t-1}) \tag{110}$$
$$= \mathrm{Cov}(X_t, C_tX_t + V_t \mid Y_{1:t-1}) \tag{111}$$
$$= \mathbb{E}\left[5(X_t - \mu_{t|t-1})(C_t(X_t - \mu_{t|t-1}) + (V_t - \mu_{v_t}))^\top \mid Y_{1:t-1}\right] \tag{112}$$
$$= \Sigma_{t|t-1}C_t^\top + \mathrm{Cov}(X_t, V_t \mid Y_{1:t-1}) \tag{113}$$
$$= \Sigma_{t|t-1}C_t^\top \quad \text{(since } X_t \text{ and } V_t \text{ uncorrelated)} \tag{114}$$

Putting these all together, we get

$$\mu_{t|t} = \mu_{t|t-1} - \Sigma_{t|t-1}C_t^\top\left(C_t\Sigma_{t|t-1}C_t^\top + R_t\right)^{-1} \tag{115}$$
$$\cdot\left(y_t - \left(C_t\mu_{t|t-1} + \mu_{v_t}\right)\right) \tag{116}$$

$$\boxed{\begin{gathered}\mu_{t|t} = \mu_{t|t-1} - K\left(y_t - (C_t\mu_{t|t-1} + \mu_{v_t})\right) \text{ (different from KF)} \\ K = \Sigma_{t|t-1}C_t^\top\left(C_t\Sigma_{t|t-1}C_t^\top + R_t\right)^{-1}\end{gathered}}$$

Covariance update

$$\Sigma_{t|t} = \Sigma_{t|t-1} - \Sigma_{xy}\Sigma_y^{-1}\Sigma_{yx} \tag{117}$$

$$\Sigma_{t|t} = \Sigma_{t|t-1} - \Sigma_{t|t-1}C_t^\top \left(C_t\Sigma_{t|t-1}C_t^\top + R_t\right)^{-1} C_t\Sigma_{t|t-1} \tag{118}$$

$$\boxed{\Sigma_{t|t} = \Sigma_{t|t-1} - KC_t\Sigma_{t|t-1} \text{ (same as KF)}}$$

Let us now apply the result from (b) to a simulation environment. **You are given the same simulation environment as the drone in HW3.** Let $p_t \in \mathbb{R}^2$ be its position, and $s_t \in \mathbb{R}^2$ its velocity. Let's model its dynamics as a discrete time double integrator perturbed by noise,

$$P_{t+1} = P_t + \delta t S_t \tag{119}$$
$$S_{t+1} = S_t + \delta t u_t + W_t, \tag{120}$$

where $u_t \in \mathbb{R}^2$ is the acceleration commanded by a remote human pilot, and δt is the discrete time step. The drone makes position measurements with GPS according to the noisy measurement model

$$Y_t = P_t + V_t. \tag{121}$$

The processes W_t, V_t are Gaussian white noise with means μ_W and μ_V, covariances Q and R, respectively, uncorrelated with each other and uncorrelated with the initial state. Let $\delta t = 1s$, $Q = I_2 m^2/s^2$, $R = 9\mathrm{I}_2 m^2$, $\mu_W = [0.3, 0.3]^\top m/s$, $\mu_V = [400, 400]^\top m$, $u_t = -2.5[\cos(0.05t) \;\; \sin(0.05t)]^T m/s^2$.
Hint: You can use the `numpy.random.multivariate_normal()` *function to create the noise.*
Note: For consistency in solutions, seed your random variables using `np.random.seed(273)`

(c) Suppose your initial guess about the state of the drone has a mean $\mu_0 = [1500m \;\; 100m \;\; 0m/s \;\; 55m/s]^T$, and covariance matrix $\Sigma_0 = [250000\mathrm{I}m^2 \;\; 0; \;\; 0 \;\; \mathrm{I}m^2/s^2]$. Use the Kalman filter equations you derived in (b) to find your estimated state trajectory. As in PS3, use your error ellipse script from PS2 to plot the means and error ellipses of the position estimate over the top of the true trajectory.

Solution: The important changes to your code here compared to the HW question is accounting for the μ_W and μ_V. These have to be accounted for in the true process noise that accounts for the true dynamics of X, the true measurements Y, and in your Kalman filter dynamics and measurement steps. The resultant figures should look very similar to what you had in your homework. The difference here though, *is that the measurements have a large bias in them.* This is shown in Figure 1.

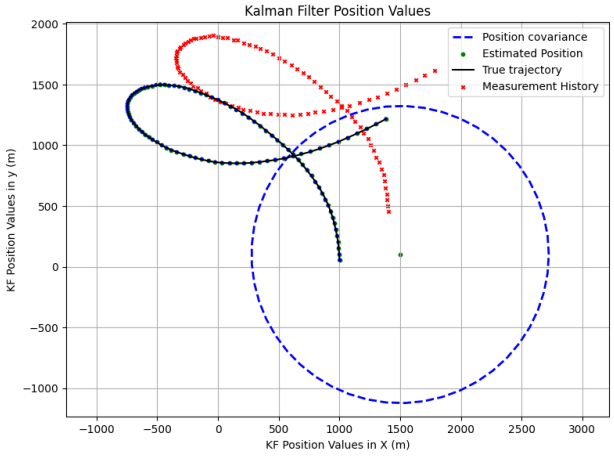


Figure 1: KF has very good estimation despite measurements that have a large bias in them.

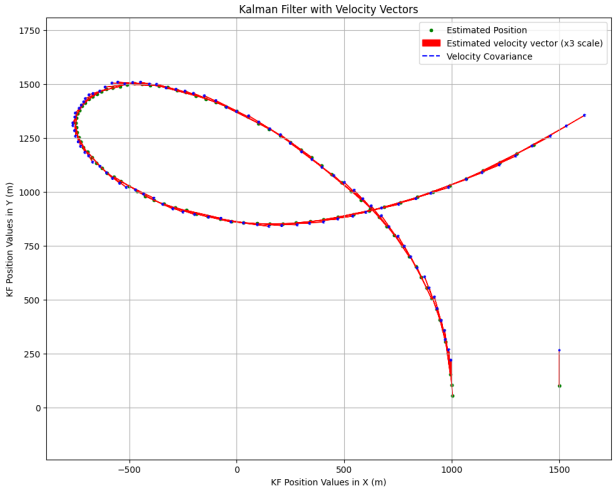


Figure 2: KF velocity estimates

Code solution:

```
    % \begin{verbatim}
############################
# Simulate the quadrotor
############################


import numpy as np
import matplotlib.pyplot as plt
from scipy.linalg import sqrtm

class QuadrotorSimulator:
    # A simulator for the quadrotor dynamics and sensor measurements

    def __init__(self, sensor,
                 mu_W = np.zeros(2),
                 mu_V = np.zeros(2),
                 q_mat= np.identity(2),
                 r_mat = 9 * np.identity(2),
                 dt = 1,
                 rng_seed = 273):
        """
        Set up the quadrotor simulator.

        Parameters
        ----------
        sensor : str
            Selection of "GPS" or "Velocity" sensor
        q_mat : np.ndarray
            The process noise covariance matrix, Q (m^2/s^2)
        r_mat : np.ndarray
            The measurement noise covariance matrix, R (m^2)
        dt : float
            The time step for the simulation (s)
        rng_seed : int
            The random seed for the simulation for reproducibility
        """
        assert sensor in ["GPS", "Velocity"], \
            "Sensor must be 'GPS' or 'Velocity'"
        self.sensor = sensor

        # Check that the matrices are square and positive definite.
        assert q_mat.shape[0] == q_mat.shape[1], "Q matrix must be square"
        assert r_mat.shape[0] == r_mat.shape[1], "R matrix must be square"
        assert np.all(np.linalg.eigvals(q_mat) > 0), \
            "Q matrix must be positive definite"
        assert np.all(np.linalg.eigvals(r_mat) > 0), \
            "R matrix must be positive definite"

        self.mu_W = mu_W
        self.mu_V = mu_V
        self.q_mat = q_mat
        self.r_mat = r_mat
```

```
53            self.dt = dt
54            self.rng_seed = rng_seed
55
56        def default_control(self, t):
57            """
58            This is the quadrotor control defined in the problem (units are
59            m/s^2).
60
61            Parameters
62            ----------
63            t : float
64                The current time (s)
65
66            Returns
67            -------
68            np.ndarray
69                The control input
70            """
71            return -2.5 * np.array([np.cos(0.05 * t), np.sin(0.05 * t)])
72
73        def noiseless_dynamics_step(self, p, s, t, u=None):
74            """
75            Simulate one step of the quadrotor dynamics without noise.
76
77            Parameters
78            ----------
79            p : np.ndarray
80                The current position of the quadrotor (m)
81            s : np.ndarray
82                The current velocity of the quadrotor (m/s)
83            t : float
84                The current time (s)
85            u : function handle
86                Function that returns the control input at time t
87
88            Returns
89            -------
90            np.ndarray
91                The new position of the quadrotor
92            np.ndarray
93                The new velocity of the quadrotor
94            """
95            # Get the current control input. If none is provided, use the default
96            if u is None:
97                u = self.default_control
98            u_curr = u(t)
99            # Check the size of the control input
100           assert u_curr.shape == (2,), \
101               f"Control input must be 2D, got {u_curr.shape}"
102
103           # Check the size of the position and velocity
104           assert p.shape == (2,), f"Position must be 2D, got {p.shape}"
105           assert s.shape == (2,), f"Velocity must be 2D, got {s.shape}"
106
107           # Compute the new position and velocity
108           p_new = p + self.dt * s
109           v_new = s + self.dt * u_curr
110
111           return p_new, v_new, u_curr
112
113       def noiseless_gps_step(self, p):
114           """
115           Simulate one step of the GPS sensor measurement without noise.
116
117           Parameters
118           ----------
119           p : np.ndarray
120               The current position of the quadrotor (m)
121
122           Returns
123           -------
124           np.ndarray
125               The sensor measurement
126           """
127           # The noiseless GPS sensor measurement is the quadrotor position
128           return p
129
130       def noiseless_velocity_step(self, s):
131           """
132           Simulate one step of the velocity sensor measurement without noise.
133
134           Parameters
135           ----------
136           s : np.ndarray
```

```
137                 The current velocity of the quadrotor (m/s)
138
139             Returns
140             -------
141             np.ndarray
142                 The sensor measurement
143             """
144             # The noiseless velocity sensor measurement is the quadrotor velocity
145             return s
146
147         def noiseless_measurement_step(self, p, s):
148             """
149             Select the appropriate sensor measurement based on the sensor type.
150             """
151             # Check the size of the position and velocity
152             assert p.shape == (2,), "Position must be 2D."
153             assert s.shape == (2,), "Velocity must be 2D."
154
155             if self.sensor == "GPS":
156                 return self.noiseless_gps_step(p)
157             elif self.sensor == "Velocity":
158                 return self.noiseless_velocity_step(s)
159             else:
160                 raise NotImplementedError(f"{self.sensor} sensor is not supported.")
161
162         def noisy_dynamics_step(self, p, s, t, u=None):
163             """
164             Simulate one step of the quadrotor dynamics with noise (see
165             self.noiseless_dynamics_step for more details).
166             """
167             noiseless_p, noiseless_s, noiseless_u = \
168                 self.noiseless_dynamics_step(p, s, t, u)
169
170             # Add process noise (only to the velocity)
171             w_noise = np.random.multivariate_normal(self.mu_W, self.q_mat) # NOTE: Non-zero mean
172
173             return noiseless_p, noiseless_s + w_noise, noiseless_u
174
175         def noisy_gps_step(self, p):
176             """
177             Simulate one step of the GPS sensor measurement with noise (see
178             self.noiseless_gps_step for more details).
179             """
180             noiseless_gps = self.noiseless_gps_step(p)
181
182             # Add measurement noise (to the position measurement)
183             v_noise = np.random.multivariate_normal(self.mu_V, self.r_mat) # NOTE: Non-zero mean
184
185             return noiseless_gps + v_noise
186
187         def noisy_velocity_step(self, s):
188             """
189             Simulate one step of the velocity sensor measurement with noise (see
190             self.noiseless_velocity_step for more details).
191             """
192             noiseless_velocity = self.noiseless_velocity_step(s)
193
194             # Add measurement noise (to the velocity measurement)
195             v_noise = np.random.multivariate_normal(self.mu_V, self.r_mat)
196
197             return noiseless_velocity + v_noise
198
199         def noisy_measurement_step(self, p, s):
200             """
201             Select the appropriate sensor measurement based on the sensor type.
202             """
203             # Check the size of the position and velocity
204             assert p.shape == (2,), "Position must be 2D."
205             assert s.shape == (2,), "Velocity must be 2D."
206
207             if self.sensor == "GPS":
208                 return self.noisy_gps_step(p)
209             elif self.sensor == "Velocity":
210                 return self.noisy_velocity_step(s)
211             else:
212                 raise NotImplementedError(f"{self.sensor} sensor is not supported.")
213
214         def simulate(self, p0, s0, num_t, rng_seed=None, u=None, noisy=True):
215             """
216             Simulate the quadrotor dynamics and sensor measurements for `num_t`
217             steps.
218
219             Parameters
220             ----------
```

```
        p0 : np.ndarray
            The initial position of the quadrotor (m)
        s0 : np.ndarray
            The initial velocity of the quadrotor (m/s)
        num_t : int
            The number of time steps to simulate
        rng_seed : int
            The random seed to use for the simulation
        u : function handle
            Function that returns the control input at time t (use None for
            the default control given in the problem)
        noisy : bool
            Whether to include noise in the simulation

        Returns
        -------
        np.ndarray
            The true position of the quadrotor at each time step
        np.ndarray
            The true velocity of the quadrotor at each time step
        np.ndarray
            The sensor measurements at each time step
        np.ndarray
            The control inputs at each time step
        """
        # Set the random seed for reproducibility
        if rng_seed is None:
            np.random.seed(self.rng_seed)
        else:
            np.random.seed(rng_seed)

        # Initialize the histories
        true_positions_history = []
        true_velocities_history = []
        measurements_history = []
        true_controls_history = []

        # Set the initial position and velocity to the current values
        p_curr = np.array(p0)
        s_curr = np.array(s0)

        for t_ind in range(num_t):
            # Get the current time
            t = t_ind * self.dt

            if noisy:
                p_curr, s_curr, u_curr = self.noisy_dynamics_step(
                    p_curr, s_curr, t, u)
                y_curr = self.noisy_measurement_step(p_curr, s_curr)
            else:
                p_curr, s_curr, u_curr = self.noiseless_dynamics_step(
                    p_curr, s_curr, t, u)
                y_curr = self.noiseless_measurement_step(p_curr, s_curr)

            # Save the current values
            true_positions_history.append(p_curr)
            true_velocities_history.append(s_curr)
            measurements_history.append(y_curr)
            true_controls_history.append(u_curr)

        # Return the histories as numpy arrays
        return np.array(true_positions_history), \
               np.array(true_velocities_history), \
               np.array(measurements_history), \
               np.array(true_controls_history)

    def plot_position_history(self, positions, measurements=None,
                              measurement_size=10, show_plot=True):
        """
        Plot the position history and the sensor measurements (if "GPS").
        The result is one figure showing one rollout.

        Parameters
        ----------
        positions : np.ndarray
            The position of the quadrotor at each time step.
        measurements : np.ndarray
            The sensor measurements at each time step.
            Set to None to only plot the position history.
        measurement_size : int
            The size of the scatter points for the measurements
        show_plot : bool
            Whether to display the plot

```

```
        Returns
        -------
        matplotlib.figure.Figure
            The figure containing the plot
        """
        # Check that we have only one run
        assert positions.ndim == 2, \
            "Data contains multiple runs. Must have 2 dimensions, " \
            f"but got {positions.ndim} dimensions."

        # Plot the position history and the sensor measurements
        fig, ax = plt.subplots(figsize=(8, 6))
        plt.title("Position History")
        plt.xlabel("x (m)")
        plt.ylabel("y (m)")

        # Plot the initial position with a black star at the highest z-order
        plt.scatter(positions[0, 0], positions[0, 1],
                    color='black', marker='*', s=100, zorder=3,
                    label="Initial Position")

        # Plot the position history
        plt.plot(positions[:, 0], positions[:, 1], label="Position")

        # Reset the color cycle for the measurements
        ax.set_prop_cycle(None)
        # Plot the sensor measurements if using GPS
        if self.sensor == "GPS" and measurements is not None:
            plt.scatter(measurements[:, 0], measurements[:, 1],
                        s=measurement_size,
                        label="GPS Measurements")

        plt.legend()
        plt.grid(True)
        plt.axis('equal')
        plt.tight_layout()

        if show_plot:
            plt.show()

        return fig


    def plot_velocity_history(self, velocities, measurements=None,
                              measurement_size=10, show_plot=True):
        """
        Plot the velocity history and the sensor measurements (if
        "Velocity"). The result is one figure showing one rollout.

        Parameters
        ----------
        velocities : np.ndarray
            The velocity of the quadrotor at each time step.
        measurements : np.ndarray
            The sensor measurements at each time step.
            Set to None to only plot the velocity history.
        measurement_size : int
            The size of the scatter points for the measurements
        show_plot : bool
            Whether to display the plot

        Returns
        -------
        matplotlib.figure.Figure
            The figure containing the plot
        """
        # Check that we have only one run
        assert velocities.ndim == 2, \
            "Data contains multiple runs. Must have 2 dimensions, " \
            f"but got {velocities.ndim} dimensions."

        # Plot the velocity history and the sensor measurements
        fig, ax = plt.subplots(figsize=(8, 6))
        plt.title("Velocity History")
        plt.xlabel("$v_x$ (m/s)")
        plt.ylabel("$v_y$ (m/s)")

        # Plot the initial velocity with a black star at the highest z-order
        plt.scatter(velocities[0, 0], velocities[0, 1],
                    color='black', marker='*', s=100, zorder=3,
                    label="Initial Velocity")

        # Plot the velocity history
        plt.plot(velocities[:, 0], velocities[:, 1], label="Velocity")
```

```
389
390         # Reset the color cycle for the measurements
391         ax.set_prop_cycle(None)
392         # Plot the sensor measurements if using Velocity
393         if self.sensor == "Velocity" and measurements is not None:
394             plt.scatter(measurements[:, 0], measurements[:, 1],
395                         s=measurement_size,
396                         label="Velocity Measurements")
397
398         plt.legend()
399         plt.grid(True)
400         plt.axis('equal')
401         plt.tight_layout()
402
403         if show_plot:
404             plt.show()
405
406         return fig
407
408     def plot_control_history(self, controls, show_plot=True):
409         """
410         Plot the control history. The result is one figure showing one rollout.
411
412         Parameters
413         ----------
414         controls : np.ndarray
415             The control input at each time step.
416         show_plot : bool
417             Whether to display the plot
418
419         Returns
420         -------
421         matplotlib.figure.Figure
422             The figure containing the plot
423         """
424         # Check that we have only one run
425         assert controls.ndim == 2, \
426             "Data contains multiple runs. Must have 2 dimensions, " \
427             f"but got {controls.ndim} dimensions."
428
429         # Plot the control history
430         fig, ax = plt.subplots(figsize=(8, 6))
431         plt.title("Control History")
432         plt.xlabel("$u_x$ (m/s$^2$)")
433         plt.ylabel("$u_y$ (m/s$^2$)")
434
435         # Plot the initial control with a black star at the highest z-order
436         plt.scatter(controls[0, 0], controls[0, 1],
437                     color='black', marker='*', s=100, zorder=3,
438                     label="Initial Control")
439
440         # Plot the control history
441         plt.plot(controls[:, 0], controls[:, 1], label="Control")
442
443         plt.legend()
444         plt.grid(True)
445         plt.axis('equal')
446         plt.tight_layout()
447
448         if show_plot:
449             plt.show()
450
451         return fig
452
453     def plot_2D_errors(self, true_vals, comparison_vals,
454                        err_cov=None,
455                        title="2D Errors",
456                        x_label="Error in x (m)",
457                        y_label="Error in y (m)",
458                        measurement_size=10, show_plot=True):
459         """
460         Plot the 2D errors between the true values and the comparison values.
461
462         Parameters
463         ----------
464         true_vals : np.ndarray
465             The true values to compare against.
466         comparison_vals : np.ndarray
467             The comparison values to plot the errors.
468         err_cov : np.ndarray
469             The error covariance matrix for the comparison values.
470         title : str
471             The title of the plot
472         measurement_size : int
```

```
            The size of the scatter points for the measurements
        show_plot : bool
            Whether to display the plot

        Returns
        -------
        matplotlib.figure.Figure
            The figure containing the plot
        """
        # Check that the dimensions match
        assert true_vals.shape == comparison_vals.shape, \
            "True and comparison values must have the same shape."

        # Compute the errors
        errors = comparison_vals - true_vals

        # Plot the errors
        fig, ax = plt.subplots(figsize=(8, 6))
        plt.title(title)
        plt.xlabel(x_label)
        plt.ylabel(y_label)

        # If we have an error covariance, plot the error ellipse (note that
        # we only plot axis-aligned ellipses, students can modify this to
        # plot more general error ellipse with their code from HW2)
        if err_cov is not None:
            assert err_cov.shape == (2, 2), \
                "Error covariance must be a 2x2 matrix."
            assert np.isclose(err_cov[1, 0], 0.0), \
                f"Error covariance must be diagonal, but got {err_cov}." + \
                "Students can modify this to plot general error ellipses."

            # Compute the error ellipse for 1 and 2 standard deviations
            for std_dev, std_ls in zip([1, 2], ['--', ':']):
                width = 2 * std_dev * np.sqrt(err_cov[0, 0])
                height = 2 * std_dev * np.sqrt(err_cov[1, 1])

                error_ellipse = plt.matplotlib.patches.Ellipse(
                    xy=[0, 0], width=width, height=height,
                    edgecolor='black', facecolor='none', linestyle=std_ls,
                    label=f"{std_dev} Std Dev Error Ellipse")
                ax.add_patch(error_ellipse)

        # Plot the errors
        plt.scatter(errors[:, 0], errors[:, 1],
                    s=measurement_size, marker='x', color='red',
                    label="Errors")

        plt.legend()
        plt.grid(True)
        plt.axis('equal')
        plt.tight_layout()

        if show_plot:
            plt.show()

        return fig
# %%


class KalmanFilter:
    def __init__(self, A, B, C, mu0, sigma0,
                 mu_W = np.zeros(4),
                 mu_V = np.zeros(2),
                 q_mat= np.identity(4),
                 r_mat = np.identity(2)) -> None:

        # Check that the matrices are square and positive definite.
        assert q_mat.shape[0] == q_mat.shape[1], "Q matrix must be square"
        assert r_mat.shape[0] == r_mat.shape[1], "R matrix must be square"
        assert np.all(np.linalg.eigvals(q_mat) > 0), \
            "Q matrix must be positive definite"
        assert np.all(np.linalg.eigvals(r_mat) > 0), \
            "R matrix must be positive definite"
        self.A = A
        self.B = B
        self.C = C
        self.mu_W = mu_W
        self.mu_V = mu_V
        self.q_mat = q_mat
        self.r_mat = r_mat
        self.mu = mu0
        self.sigma = sigma0
        pass
```

```
557
558     def predict(self, u):
559         self.mu_predict = self.A @ self.mu + self.B @ u + self.mu_W # NOTE: Addition of mu_W here.
560         self.sigma_predict = self.A @ self.sigma @ self.A.T + self.q_mat
561
562         return self.mu_predict, self.sigma_predict
563
564     def update(self, y):
565         K = self.sigma_predict @ self.C.T @ np.linalg.inv(self.C @ self.sigma_predict @ self.C.T +
                self.r_mat)
566         self.mu = self.mu_predict + K @ (y - self.C @ self.mu_predict - self.mu_V) # NOTE: Addition of
                mu_V here
567         self.sigma = self.sigma_predict - K @ self.C @ self.sigma_predict
568         return self.mu, self.sigma
569
570
571 # Calculates the standard gaussian circle
572 P = 0.95
573 theta = np.linspace(0, 2*np.pi, 50)
574 r = np.sqrt(-2*np.log(1 - P))
575 w1 = r*np.cos(theta)
576 w2 = r*np.sin(theta)
577
578 def ellipse_points(mu, cov):
579     theta_len = len(theta)
580     w = np.zeros((theta_len, 2, 1))
581     x = np.zeros((theta_len, 2, 1))
582     for i in range(theta_len):
583         w[i] = np.array([[w1[i]], [w2[i]]])
584         x[i] = sqrtm(cov) @ w[i] + mu.reshape((2,1))
585     return x
586
587 def plot_mean_and_cov_traj(mean_vals,
588                     cov_vals,
589                     true_vals,
590                     y_vals,
591                     title="Kalman Filter Position Values",
592                     x_label="KF Position Values in X (m)",
593                     y_label="KF Position Values in y (m)",
594                     measurement_size=10, show_plot=True):
595
596     # Plot the means
597     n = mean_vals.shape[0]
598     fig, ax = plt.subplots(figsize=(8, 6))
599     plt.title(title)
600     plt.xlabel(x_label)
601     plt.ylabel(y_label)
602
603     # Plot the covariances
604     for i in range(n):
605         ellipse = ellipse_points(mean_vals[i, :], cov_vals[i,:,:])
606         if i == 0:
607             plt.plot(ellipse[:,0, 0], ellipse[:,1, 0], 'b--', linewidth=2, label='Position covariance'
                    )
608         else:
609             plt.plot(ellipse[:,0, 0], ellipse[:,1, 0], 'b--', linewidth=2)
610
611     # Plot the means
612     plt.scatter(mean_vals[:, 0], mean_vals[:, 1],
613                 s=measurement_size, marker='o', color='g', label='Estimated Position')
614
615     plt.plot(true_vals[:, 0], true_vals[:, 1], color='k', label='True trajectory')
616
617     # Plt the measurement
618     plt.scatter(y_vals[:, 0], y_vals[:, 1],
619                 s=measurement_size, marker='x', color='r', label='Measurement History')
620
621     plt.legend()
622     plt.grid(True)
623     plt.axis('equal')
624     plt.tight_layout()
625
626     if show_plot:
627         plt.show()
628     return fig
629
630
631 def plot_mean_and_cov_traj_with_vel(mean_vals,
632                             vel_means,
633                             vel_covs,
634                             title="Kalman Filter Position and Velocity",
635                             x_label="Position X (m)",
636                             y_label="Position Y (m)",
637                             measurement_size=10,
```

```
                                show_plot=True):
    n = mean_vals.shape[0]
    fig, ax = plt.subplots(figsize=(10, 8))
    plt.title(title)
    plt.xlabel(x_label)
    plt.ylabel(y_label)

    # Plot position means
    plt.scatter(mean_vals[:, 0], mean_vals[:, 1],
                s=measurement_size, marker='o', color='g', label='Estimated Position')

    scale = 3.0
    # Plot velocity vectors and their covariances
    for i in range(n):
        start_point = mean_vals[i, :]
        velocity = vel_means[i, :]

        # Draw velocity vector
        if i == 0:
            plt.arrow(start_point[0], start_point[1],
                  scale*velocity[0], scale*velocity[1],
                  head_width=0.2, head_length=0.3, fc='r', ec='r', label='Estimated velocity vector (
                      x3 scale)')
        else:
            plt.arrow(start_point[0], start_point[1],
                  scale*velocity[0], scale*velocity[1],
                  head_width=0.2, head_length=0.3, fc='r', ec='r')

        # Compute ellipse at the tip of the velocity vector
        vel_tip = start_point + scale*velocity
        vel_ellipse = ellipse_points(vel_tip, vel_covs[i,:,:])
        if i == 0:
            plt.plot(vel_ellipse[:,0,0], vel_ellipse[:,1,0], 'b--', linewidth=1.5, label='Velocity
                Covariance')
        else:
            plt.plot(vel_ellipse[:,0,0], vel_ellipse[:,1,0], 'b--', linewidth=1.5)

    plt.legend()
    plt.grid(True)
    plt.axis('equal')
    plt.tight_layout()

    if show_plot:
        plt.show()
    return fig

# Define simulation parameters
n_steps = 100
p0 = [1000, 0]
s0 = [0, 50]

# Initialize the simulator
Q = np.eye(2)
R = 9*np.eye(2)
mu_W = np.array([0.3, 0.3])
mu_V = np.array([400, 400])
quadrotor = QuadrotorSimulator(sensor="GPS", mu_W=mu_W, mu_V=mu_V, q_mat=Q, r_mat=R)

# Do a single run
p0 = np.array([1000.0, 0.0]).T
s0 = np.array([0.0, 50.0]).T
phist, shist, yhist, uhist = quadrotor.simulate(p0, s0, n_steps)

# %% Interfacing with Kalman Filter implementation
# %% Question 4
dt = 1.0

# State space matrices based on Q3a
A = np.array([[1.0, 0.0, dt, 0.0],
              [0.0, 1.0, 0.0, dt],
              [0.0, 0.0, 1.0, 0.0],
              [0.0, 0.0, 0.0, 1.0]])
B = np.array([[0.0, 0.0],
              [0.0, 0.0],
              [dt, 0.0],
              [0.0, dt]])
C = np.array([[1.0, 0.0, 0.0, 0.0],
              [0.0, 1.0, 0.0, 0.0]])
mu0 = np.array([1500.0, 100.0, 0.0, 55.0]).T
sigma0 = np.hstack((np.vstack((250000.0*np.eye(2), np.zeros((2,2)))), np.vstack((np.zeros((2,2)), np.
    eye(2)))))
Q = np.diag([1e-10, 1e-10, 1.0, 1.0])
R = 9.0 * np.eye(2)
mu_W = np.array([0, 0, 0.3, 0.3])
```

```
719   mu_V = np.array([400, 400])
720   kalman_filter = KalmanFilter(A, B, C, mu0, sigma0, mu_W = mu_W, mu_V=mu_V, q_mat=Q, r_mat=R)
721   state_vals = np.zeros((n_steps+1, 4))
722   cov_vals = np.zeros((n_steps+1, 4, 4))
723   state_vals[0, :] = mu0
724   cov_vals[0, :, :] = sigma0
725
726   for t_index, (u, y) in enumerate(zip(uhist, yhist)):
727       # Use (u, y) to step your Kalman Filter
728       state_predict, cov_predict = kalman_filter.predict(u)
729       state_vals[t_index+1, :], cov_vals[t_index+1, :, :] = kalman_filter.update(y)
730       pass
731
732   pos_means = state_vals[:, 0:2]
733   vel_means = state_vals[:, 2:4]
734   pos_covs = cov_vals[:, 0:2, 0:2]
735   vel_covs = cov_vals[:, 2:4, 2:4]
736
737
738   # %% Plotting KF quantities
739   fig1 = plot_mean_and_cov_traj(pos_means, pos_covs, phist, yhist, show_plot=True)
740   fig1.savefig('./midterm_Q4c_position_with_KF.png')
741   fig2 = plot_mean_and_cov_traj_with_vel(pos_means, vel_means, vel_covs, title='Kalman Filter with
          Velocity Vectors', x_label='KF Position Values in X (m)', y_label='KF Position Values in Y (m)',
          show_plot=True)
742   fig2.savefig('./midterm_Q4c_velocity_with_KF.png')
743       % \end{verbatim}
```